

Η σειρά γραμμών αλυσίδων Αλγόριθμοι και Πεδιωδοστικά

Άσκηση 1: Υπολογισμός Κυριαρχιών Θέσεων

$A[1, \dots, n]$, $n \in \mathbb{N}$

$\forall i \geq 2$ η θέση i κυριαρχεί της θέσης i αν
η j είναι η πλησιέστερη θέση με $0 \leq j < i$
και $A[j] > A[i]$

Ο αλγόριθμος που υπολογίζει τις κυριαρχίες
θέσεων των στοιχείων του πίνακα A και
τις αποθηκεύει στον πίνακα B είναι ο ακόλουθος:

```

B[1] ← 0; stack stack.push(A[1]); lastpos ← 1;
for i from 2 to n do
    if  $A[i] > \text{stack.top()}$  then {
        while  $A[i] > \text{stack.top()}$  and not stack.empty
        {last ← stack.pop();}
         $B[i] \leftarrow B[\text{last}];$   $\text{stack.push}(A[i]);$   $i \leftarrow i + 1;$ 
        lastpos ← 1;
    }
else
     $B[i] \leftarrow \text{stack.top().position};$   $\text{lastpos};$ 
     $\text{stack.push}(A[i]);$ 
     $\text{lastpos} \leftarrow i;$ 

```

Ο αλγόριθμος αυτός, όταν το τρέχων στοιχείο είναι μικρότερο από την κορυφή της στοίβας διατρέχει μία φορά τον πίνακα και κάποιες φορές τα περιεχόμενα της στοίβας. Κάθε στοιχείο όπως του πίνακα Α μπορεί να βρει το ποσό μία φορά στη στοίβα και συγχεριμένα με τη σειρά των που έχει στον πίνακα, καθώς και μία φορά μόνο μπορεί να βγει από αυτήν. Επομένως η χρονική πολυπλοκότητα του αλγόριθμου είναι $O(n)$.

Αναφορικά με την ορθότητα του αλγόριθμου, γνωρίζουμε ότι η κυρίαρχη θέση μιας θέσης στον πίνακα Α είναι η κορυφή, προηγούμενη και μεγαλύτερη από αυτήν. Αυτό εφαστραδίζεται στον αλγόριθμο, αφού αν το τρέχων στοιχείο είναι μικρότερο από την κορυφή ~~πλην~~, η κορυφή αυτή θα είναι η κυρίαρχη θέση του, ενώ αν το τρέχων στοιχείο είναι μεγαλύτερο από την κορυφή, ο αλγόριθμος ψάχνει μέσα στη στοίβα ~~ως~~ μεγαλύτερα από τα προηγούμενα στοιχεία μέχρι είτε να βρει κάποιο μεγαλύτερο, είτε να αδειάσει η στοίβα, το οποίο σημαίνει πως η κυρίαρχη θέση του τρέχοντος στοιχείου είναι το 0. Επομένως ο αλγόριθμος είναι ορθός.

Άσκηση 2: Επιλογή

a) Πόλυμνοδο S με n θετικούς ακεραίους
 $\forall i \in S \quad i \leq M$

$$F_S(i) = |\{x \in S : x \leq i\}|$$

Input: $K \in \mathbb{N}$, $1 \leq K \leq n$

Output: $m \in \mathbb{N}$, m το K -οστό μικρότερο στοιχείο του S .

Για την εύρεση του K -οστού μικρότερου στοιχείου του S θα χρησιμοποιήσουμε ένα συνδυασμό διαδικασίας αναζήτησης και αναζήτησης με παρεμβολή. Θα αξιοποιήσουμε την ταχύτητα της αναζήτησης με παρεμβολή στην περίπτωση σχεδόν ομοιόμορφης ή προβλέψιμης κατανομής, ενώ θα χρησιμοποιήσουμε διαδικασί αναζήτησης σε οποιαδήποτε άλλη περίπτωση.

Το K -οστό μικρότερο στοιχείο του S είναι το ελάχιστο στοιχείο $\min$, για το οποίο ισχύει $F_S(\min) = K$.

Υπολογίζουμε την αναμενόμενη θέση ως εξής:
$$pos = 1 + \frac{(K - F_S(1))(M - 1)}{F_S(M) - F_S(1)}$$
 και αν το pos

δεν ανήκει στο $[1, M]$ τότε κάνουμε διαδικασί αναζήτησης

```

low ← 1; up ← M;
while low ≤ up do
    if  $F_s(\text{up}) \neq F_s(\text{low})$  then
         $\text{pos} \leftarrow \text{low} + (\text{K} + F_s(\text{low})) * (\text{up} - \text{low}) //$ 
             $// F_s(\text{up}) - F_s(\text{low});$ 
    else
         $\text{pos} \leftarrow (\text{low} + \text{up}) // 2;$ 

    if  $\text{pos} < \text{low}$  or  $\text{pos} > \text{high}$  then
         $\text{pos} \leftarrow (\text{low} + \text{up}) // 2;$ 

```

```

if  $F_s(\text{pos}) \geq K$  then
    return (pos)

```

```

else if  $F_s(\text{pos}) \geq K$  then
    up ← pos - 1;

```

```

else

```

```

    low ← pos + 1;

```

```

return (low);

```

Ο αλγόριθμος αυτός, αφού επιλέξει το κατάλληλο σημείο αναζήτησης, καλεί την F_s για το σημείο αυτό. Αν είναι μικρότερο το αποτέλεσμα από K τότε σίγουρα το K -οστό στοιχείο θα βρίσκεται πιο δεξιά από το σημείο αναζήτησης. Αν είναι μεγαλύτερο, η αμοή και ίσο, παλι θα φάιντοτε αριστερά από το σημείο αναζήτησης, μέχρι το low να γίνει μεγαλύτερο από το up (συμμερίζεται $\text{up} + 1$). Τότε გίμαστε σίγουροι πως βρήκατε το ελάχιστο στοιχείο min για το οποίο $F_s(\text{min}) = K$. Επομένως ο αλγόριθμος είναι ορθός.

Το πλήθος των απαιτούμενων υδρήσεων στην F_S είναι $O(\log \log M)$ αν έχουμε κατάλληλη κατανομή και $O(\log M)$ στη χειρότερη περίπτωση, αν κάνουμε δηλαδή διαδοχική αναζήτηση.

β) $A[1 \dots n]$ πίνακας διακριτικών ^{θετικών} ακεραίων
Μ το μέγιστο στοιχείο του A

Ποδυσυνόδο S αποτελείται από όδες τις h αρνητικές διαφορές ζευγών στοιχείων του A .

$$S = \{ A[i] - A[j] : i \neq j \text{ και } A[i] > A[j] \}$$

Προφανώς το μόνο που μένει να κάνουμε είναι για αποδοτική F_S και να εφαρμόσουμε τον αλγόριθμο του αλγορίθμου.

Αυτό συμβαίνει διότι:

- Το μέγιστο στοιχείο του A είναι το M και αποτελείται από θετικούς ακεραίους, επομένως το μέγιστο στοιχείο του ποδυσυνόδου S θα είναι το M .
- Το πλήθος των στοιχείων του ποδυσυνόδου S ισούται με το πλήθος των στοιχείων διαφορών ζευγών στοιχείων του A για τα οποία $i \neq j$ και $A[i] > A[j]$. Υπάρχουν $n-1$ μεγαλύτερα στοιχεία από το μεγαλύτερο, $n-2$ από το 2ο μεγαλύτερο κλπ. Επομένως το πλήθος των στοιχείων του ποδυσυνόδου S είναι $\frac{n(n-1)}{2}$.

Για την υλοποίηση της F_S καταρχήν θα ταξινομήσουμε τον πίνακα A για να βρίσκουμε εύκολα τα j τ.ω $A[i] > A[j] \forall i$. Αυτό θα στοιχίσει $n \log n$ σε χρόνο.

Υστερα θα ακολουθήσουμε τον εξής αλγόριθμο:

```
elements ← 0; F_S(1)  
while  $i \leq n$  do (using binary search) ↗  
    find smallest  $j$  such that  $1 \leq j < i$  and  $A[i] - A[j] \geq 1$   
    elements ← elements + (i - j);  
     $i \leftarrow i + 1$ ;  
return (elements);
```

Στη συνέχεια για την εύρεση του k -οστού μικρότερου στοιχείου του S θα ακολουθείται ο αλγόριθμος του α) ερωτήματος.

Η υπολογιστική πολυπλοκότητα του αλγορίθμου είναι:

- $O(n \log n)$ για την ταξινόμηση του πίνακα A
- $O(n \log n \log M)$ για τον αλγόριθμο του α) ερωτήματος με αυτή την υλοποίηση της F_S και $O(n \log n \log \log M)$ αν πετύχει η αναζήτηση με παρεμβολή. Αυτό συμβαίνει διότι ο αλγόριθμος του α) ερωτήματος με κόστος $\log M$ ($\log \log M$ αντιστοίχα) καλεί καθε φορά την F_S , η οποία κοστίζει n (για τη λίστα $1 \dots n$) επί $\log n$ (για τη δυαδική αναζήτηση)

Οπότε η συνολική πολυπλοκότητα του αλγόριθμου είναι $O(n \log n + n \log n \log M) = O(n \log n \log M)$ ($O(n \log n \log \log M)$ αν πετυχεί η αναζήτηση με παρεμβολή).

Αναφορικά με την ορθότητα, για κάθε i ψάχνουμε το ελάχιστο j για το οποίο η διαφορά $|A[i] - A[j]|$ είναι ≤ 1 . Αυτό μας εφασφαλίζει ότι για το στοιχείο i θα βρεθούν όλες οι διαφορές με τιμή μικρότερη ή ίση των 1, καθώς όσο πληθαίνουν αριστερά στον πίνακα τα στοιχεία μικραίνουν και οι διαφορές μετακινούνται. Επιπλέον, προσθέτουμε στο αποτέλεσμα το πόσο απέχουν ~~οι~~ τα στοιχεία i, j μεταξύ τους, το οποίο αντιστοιχεί στο πλήθος των στοιχείων του S Επομένως ο αλγόριθμος είναι ορθός.

Άσκηση 3: Άθροισμα στοιχείων υποσυνόλων και υποσυνόλων

a) $S = \{s_1, \dots, s_n\}$, $n \geq 3$ θετικοί αμέγαλοι

$B \geq 1$, $K \geq 1$. Ψάχνουμε # υποσυνόλων $A \subseteq S$ με $|A| \leq K$

στοιχεία και $\sum_{s_i \in A} s_i = B$

Για την επίλυση αυτή του προβλήματος θα χρησιμοποιήσουμε τον αλγόριθμο του subset sum, τροποποιημένο με πίνακα 3 διαστάσεων αντί για 2,

με την 3η δίδοται να αντιπροσωπεύει το μέγεθος των υποσυνόλων. Συγκεκριμένα:

Η αναδρομική σχέση που υπολογίζει τα στοιχεία του πίνακα είναι η εξής:

$$S(i, b, m) = \begin{cases} 1, & i = b = m = 0 \\ 0, & i = 0, \text{ και } b = 1, \dots, B \text{ και } m = 1, \dots, K \\ S(i-1, b, m), & i = 1, \dots, n, b = 1, \dots, B, m = 1, \dots, K \\ & \text{και } (b < s_i \text{ και } m \leq n) \\ S(i-1, b, m) + S(i-1, b-s_i, m-1), & i = 1, \dots, n, \\ & b = 1, \dots, B, m = 1, \dots, K \text{ και } b \geq s_i \text{ και } m > 0 \end{cases}$$

Ουσιαστικά υπολογίζουμε K 2×2 πίνακες subset sum με μέγεθος $n \times B$ με στοιχεία το πλήθος διαστημάτων υποσυνόλων με άθροισμα b και μέγεθος m $\forall i = 1, \dots, n$ $\forall b = 1, \dots, B$ και στο τέλος προσθέτουμε τα K αυτών δεξιά στοιχεία του πίνακα ($S(n, B)$) των επιμέρους K 2×2 πινάκων $\forall m = 1, \dots, K$. Ο αλγόριθμος σε ψευδοκώδικα διατυπώνεται ως εξής:

~~$S[0][0][0] \leftarrow 1;$~~

~~$\text{for } b \leftarrow 1 \text{ to } B \text{ do}$~~

~~$\text{for } m \leftarrow 1 \text{ to } K \text{ do}$~~

~~$S[0, b, m] \leftarrow 0;$~~

~~$S[0, b, m] \leftarrow 0;$~~

$\text{for } i \leftarrow 1 \text{ to } n \text{ do}$

$\text{for } b \leftarrow 0 \text{ to } B \text{ do}$

$\text{for } m \leftarrow 0 \text{ to } K \text{ do}$

$S[i, b, m] \leftarrow S[i-1, b, m];$

if $b \geq s_i$ and $m > 0$ then
 $S[i, b, m] \leftarrow S[i, b, m] + S[i-1, b-s_i, m-1];$
 return $(\sum_{m=1} S[n, B, m]);$

Αναφορικά με την υπολογιστική πολυπλοκότητα του αλγόριθμου, έχουμε ένα τρίτο iteration για τα n, B, k , επομένως ο αλγόριθμος θα έχει πολυπλοκότητα $O(n \cdot B \cdot k)$.

Ο αλγόριθμος αυτός, παρόλο που έχει πολυωνυμική μορφή, δεν είναι πολυωνυμικός χρόνος. Συγκεκριμένα, είναι ψευδοπολυωνυμικός χρόνος, εφόσον εξαρτάται γραμμικά τόσο από το πλήθος των στοιχείων εισόδου n , όσο και από το μέγεθος του αθροίσματος υποσυνόλων B , καθώς και από το μέγεθος του πλήθους των υποσυνόλων k , και οι αριθμοί αυτοί μπορεί να είναι ευθέως μεγαλύτεροι από το n .

Αναφορικά με την ορθότητα του αλγόριθμου, για κάθε έναν από τους k 2×2 πίνακες, τα στοιχεία τους υπολογίζονται σωστά, καθώς σε κάθε επανάληψη προστίθενται το πλήθος των υποσυνόλων που πληρούν τις προϋποθέσεις, ξεκινώντας για $k=1$ όπου ένα στοιχείο γίνεται 1 αν $s_i = b$ και μεταφέρεται η τιμή στα αποκατασκευασμένα στοιχεία. Για κάθε k το σωστό αποτέλεσμα αποθηκεύεται στο $S(n, B)$ στοιχείο. Επομένως το πλήθος των υποσυνόλων με μέγεθος $\leq k$ είναι το άθροισμα των $S(n, B)$ των k πινάκων και ο αλγόριθμος είναι ορθός.

β) Ακολουθία $a = (a_1, \dots, a_n)$ από $n \geq 3$ θετικούς αμέραιους

$K, 1 \leq K \leq n-1$. Ψάχνουμε για K -σχεδόν γνήσιως αύξοντα υποακολουθία της a , τα στοιχεία της οποίας επιτυγχάνουν μέγιστο συνολικό άθροισμα μεταξύ όλων των K -σχεδόν γνήσιων αύξοντων υποακολουθιών της a .

Για την επίλυση αυτού του προβλήματος, θα χρησιμοποιήσουμε τον εἰς αλγόριθμο, ο οποίος χαρακτηρίζεται από την ακόλουθη αναδρομική σχέση

$$S(i, j) = \begin{cases} 0, & i=0 \\ a_1, & i=1 \\ S(i-1, 0), & j=0 \text{ και } a_{i-1} \geq a_i, i=2, \dots, n \\ S(i-1, 0) + a_i, & j=0 \text{ και } a_{i-1} < a_i, i=2, \dots, n \\ S(i-1, j) + a_i, & j \geq 1 \text{ και } a_{i-1} < a_i, i=2, \dots, n \\ \max\{S(i-1, j-1) + a_i, S(i-1, j)\}, & j \geq 1 \text{ και } a_{i-1} \geq a_i, i=2, \dots, n \end{cases}$$

Ο αλγόριθμος διατυπώνεται σε ψευδοκώδικα ως εξής:

```

for j ← 0 to K do
    S[0, j] ← 0;
    S[1, j] ← a1;
for j ← 2 to n do
    if j ≥ 1 and ai-1 < ai then
        S[i, j] ← S[i-1, j] + ai;
    else if j ≥ 1 and ai-1 ≥ ai then
        t ← S[i-1, j-1] + ai;
        if S[i-1, j] ≥ t then
            S[i, j] ← S[i-1, j];
        else
            S[i, j] ← t;
    else if j = 0 and ai-1 < ai then

```

```

 $S[i, 0] \leftarrow S[i-1, 0] + a_i;$ 
else if  $j=0$  and  $a_{i-1} \geq a_i$  then
 $S[i, 0] \leftarrow S[i-1, 0];$ 
return( $S[n, k]$ );

```

Αναφορικά με την υπολογιστική πολυπλοκότητα του αλγορίθμου, έχουμε ένα iteration από το 2 μέχρι το n και σε κάθε επανάληψη μπορεί να υπολογιστεί μέχρι k τιμές για το j . Επομένως, ο αλγόριθμος έχει υπολογιστική πολυπλοκότητα $O(n \cdot k)$.

Αναφορικά με την ορθότητα του αλγορίθμου, έχουμε ως βάση της επαγωγής πως για $i=0$ $S[0, j]=0$ και για $i=1$ $S[1, j]=a_1$. Ως επαγωγική υπόθεση θεωρούμε πως η αναδρομική σχέση υπολογίζεται σωστά το $S[i-1, j]$ για όλα τα j . Για το επαγωγικό βήμα έχουμε:

- Αν $a_{i-1} < a_i$ η υποακολουθία παρακβέλει γνησίως αύξουσα, επομένως στο μέγιστο άθροισμα προσθέτουμε το a_i και προκύπτει πάλι το μέγιστο άθροισμα, δηλαδή η βέλτιστη λύση.
- Αν $a_{i-1} \geq a_i$ έχουμε δύο επιλογές. Είτε προσθέτουμε το a_i στο άθροισμα, το οποίο μας στοιχίζει ένα j δηλαδή μια διακοπή, είτε δεν προσθέτουμε το a_i και κρατάμε αναδρομικά την $S[i-1, j]$. Από αυτές, η βέλτιστη λύση είναι αυτή με το μέγιστο άθροισμα.
- Αν $j=0$ δεν έχουμε άλλη διαθέσιμη διακοπή, επομένως προσθέτουμε μόνο τα a_i που δε διακόπτουν τη γνησίως αύξουσα ιδιότητα της ακολουθίας. Επομένως η αναδρομική σχέση επαληθεύεται πως το επιθυμητό $S[i, j]$ είναι το βέλτιστο και ο αλγόριθμος είναι ορθός.

Ασκηση 4: Μη επιυαδύπτονα διαστήματα μέγιστου
συνολικού μήκους

Έχουμε n διαστήματα $[s_1, f_1), \dots, [s_n, f_n), s_i, f_i \in \mathbb{N}$
 $s_i < f_i \ \forall i: 1 \leq i \leq n$

Θέλουμε να επιλέξουμε κάποια από τα n διαστήματα,
ώστε τα επιλεγμένα διαστήματα να μην επιυαδύπτονται
μεταξύ τους και να έχουν μέγιστο συνολικό μήκος.

1. α) Απρόστος αλγόριθμος που επιλέγει σε κάθε
επανάληψη το διαθέσιμο διάστημα με μέγιστο μήκος

Έστω $n=3$ διαστήματα: $[0, 6), [0, 5], [5, 10]$

Ο αλγόριθμος αυτός θα επιλέξει το διάστημα $[0, 6)$
και δε θα μπορέσει να επιλέξει κανένα άλλο λόγω
επιυαδύψης, επομένως δεν οδηγεί πάντα στη βέλτιστη
λύση.

β) Απρόστος αλγόριθμος που επιλέγει το διαθέσιμο
διάστημα με ελάχιστο χρόνο ολοκλήρωσης

Έστω $n=2$ διαστήματα: $[0, 2), [1, 5]$

Ο αλγόριθμος αυτός θα επιλέξει το διάστημα
 $[0, 2)$ και κανένα άλλο λόγω επιυαδύψης, επομένως
δεν οδηγεί πάντα στη βέλτιστη λύση.

2. Για την επίλυση αυτού του προβλήματος θα
χρησιμοποιήσουμε τον εξής αλγόριθμο:

Αρχικά, θα ταξινομήσουμε τα διαστήματα σε
αύξουσα σειρά με βάση το f_i . Ύστερα θα φτιάξουμε
έναν $n \times 1$ πίνακα, που θα υπολογίζει το μέγιστο
συνολικό μήκος για κάθε διάστημα με βάση την

αυόλουθη αναδρομική σχέση:

$$L(i) = \begin{cases} 0 & i=0 \\ f_1 - s_1 & i=1 \\ \max \{ L(i-1), f_i - s_i + L_{\text{ino}}(i) \} & i=2, \dots, n \end{cases}$$

Όπου: $L_{\text{ino}}(i) = L_{\text{last non-overlapping}}(i) = L(j) : f_j = \max_{k=0, \dots, i} f_k \leq s_i$
το μέγιστο του τελευταίου μη επικαλυπτόμενου διαστήματος του i .

Ο αλγόριθμος διατυπώνεται σε ψευδοκώδικα ως εξής:

```
/*  $f_1 \leq f_2 \leq \dots \leq f_n$  */ (ταξινόκηση με βάση τα  $f_i$ )  $L[0] \leftarrow 0$ ;  
 $L[1] \leftarrow f_1 - s_1$ ;  
for  $i \leftarrow 2$  to  $n$  do  
     $t \leftarrow L[i-1] + f_i - s_i$ ;  
    if  $L[i_{\text{no}}] \geq t$  then  
         $L[i] \leftarrow L[i-1]$ ;  
    else  
         $L[i] \leftarrow t$ ;  
return  $L[n]$ ;
```

Αναφορικά με την υπολογιστική πολυπλοκότητα του αλγορίθμου, έχουμε μια ταξινόκηση, η οποία κοστίζει χρόνο $O(n \log n)$. Στη συνέχεια, έχουμε ένα iteration μέχρι το n και σε κάθε επανάληψη ψάχνουμε το $L_{\text{ino}}[i]$. Η αναζήτηση για το $f_j = \max_{k=0, \dots, i} f_k \leq s_i$ μπορεί να γίνει με δυαδική αναζήτηση, με αποτέλεσμα το κόστος του iteration σε χρόνο να είναι $O(n \log n)$. Επομένως ο αλγόριθμος έχει χρονική πολυπλοκότητα $O(n \log n)$.

Αναφορικά με την ορθότητα του αλγορίθμου, ο πίνακας αρχικοποιείται με $L[0] = 0$, που τυπώνει τη βέλτιστη λύση για 0 διαστήματα. Από τα ταξινομημένα διαστήματα, επιλέγεται το πρώτο και στη συνέχεια

Ένα διάστημα επιλέγεται αν δεν είναι επικαλυπτόμενο με το προηγούμενό του και αν το μήκος του βελτιώνει το συνολικό μήκος διαστήματος. Επομένως, η λύση χτίζεται με βάση τις επιμέρους βέλτιστες λύσεις $\forall$ μικρότερο i . Εφόσον η λύση προκύπτει από το συνδυασμό των βέλτιστων λύσεων όλων των υποπροβλημάτων $\forall i=1, \dots, n$, η λύση αυτή είναι βέλτιστη και ο αλγόριθμος είναι ορθός.

Άσκηση 5: Θέσεις Σταθμεύσεων

N αυτοκίνητα που θέλουν να παρκάρουν κατά μήκος του $[0, L]$

$C_i = (r_i, d_i, s_i)$ θετικοί αμέτρητοι, με το r_i να δηλώνει αΐα, το d_i μέγιστη απόσταση από το 0 και s_i το μήκος του αυτοκινήτου.

Το αυτοκίνητο i δίνει r_i ευρώ για να παρκάρει στο διάστημα $[t, t+s_i]$ με $t+s_i \leq d_i$. Το αυτοκίνητο i δε δέχεται να παρκάρει αν $t+s_i > d_i$ ή αν το διάστημα που του δίνεται να παρκάρει έχει μήκος $< s_i$.

Ψάχνουμε ποια αυτοκίνητα θα παρκάρουν και σε ποια διαστήματα, έτσι ώστε σε κάθε σημείο του $[0, L]$ να παρκάρει το πολύ ένα αυτοκίνητο και να μεγιστοποιηθεί η συνολική αΐα των αυτοκινήτων που παρκάρουν σύμφωνα με τις επιθυμίες τους.

1. $s_i = 1 \forall i$, δηλαδή τα αυτοκίνητα έχουν όλα μοναδιαίο μήκος και αντιμετωπίζουμε το διάστημα $[0, L]$ ως σύνολο L διαθέσιμων θέσεων. Το αυτοκίνητο i συνεισφέρει αΐα r_i αν παρκάρει στη θέση $1 \leq d_i$ και αΐα 0 διαφορετικά.

Για την επίλυση του προβλήματος όπου $s_i = 1$ για κάθε αυτοκίνητο i , μπορούμε να χρησιμοποιήσουμε άπλοστο αλγόριθμο και `maxheap`. Συγκεκριμένα:

- Ταξινομούμε τα αυτοκίνητα κατά αύξουσα d_i . Σε περίπτωση ισοβαθμίας, ταξινομούμε τα αυτοκίνητα με το ίδιο d_i κατά φθίνουσα v_i .

- Επαναλαμβάνονται από το L μέχρι το 1 :

- Βάζουμε στο σωρό (`maxheap`) τα αυτοκίνητα που μπορούν να παρκάρουν στη θέση 1 , δηλαδή τα αυτοκίνητα με $d_i \geq 1$.

- Από τα διαθέσιμα αυτοκίνητα στο σωρό, επιλέγουμε το $1o$, δηλαδή αυτό με τη μεγαλύτερη αξία

- Συνεχίζουμε για τη θέση $1-1$ κλπ.

Ο αλγόριθμος διατυπώνεται σε ψευδοκώδικα ως εξής:

```
/*  $d_1 \leq d_2 \leq \dots \leq d_n$  */ (Ταξινόηση κατά αύξουσα τιμή  $d_i$   

 $C \leftarrow \emptyset$ ; και κατά φθίνουσα τιμή  $v_i$  σε  

 $\text{maxheap} \leftarrow \emptyset$ ; περίπτωση ισοβαθμίας  $d_i$ )  

 $i \leftarrow 1$ ;  

for  $l \leftarrow L$  to  $1$  do  

    while  $i \leq N$  and  $d_i \geq l$  do  

         $\text{maxheap} \leftarrow \text{maxheap} \cup \{C_i : d_i \geq l\}$ ;  

         $i \leftarrow i + 1$ ;  

    if  $\text{maxheap} \neq \emptyset$  then  

         $C \leftarrow C \cup \text{maxheap}[1]$ ;  

return  $C$ ;
```

Αναφορικά με την υπολογιστική πολυπλοκότητα του αλγόριθμου, έχουμε μια ταξινόηση που κοστίζει $O(N \log N)$, N εισαγωγές στο σωρό που κοστίζουν $\log N$ η κάθε μια, άρα $O(N \log N)$ για το σωρό και ένα iteration από το L μέχρι το 1 , που κοστίζει $O(1)$. Επομένως,

Ο αλγόριθμος έχει υπολογιστική πολυπλοκότητα $O(N \log N + L)$.

Αναγκαστικά με την ορθότητα του αλγορίθμου, θα την αποδείξουμε με βάση το επιχείρημα ανταλλαγής.

Έστω C^* βέλτιστη λύση και C η λύση που προτείνει το απρόσκοπτο κριτήριο. Υποθέτουμε ότι $C^* \neq C$ σε ένα αυτοκίνητο, C_j η επιλογή της βέλτιστης στη θέση 1 και C_k η επιλογή του απρόσκοπτου στη θέση 1. Προφανώς, για να μπορέσω να παραδώσω τα C_j, C_k στη θέση 1 θα πρέπει $d_j \geq 1$ και $d_k \geq 1$. Ο απρόσκοπτος θα επιλέξει από τα διαθέσιμα αυτοκίνητα στο σωρό αυτό με τη μεγαλύτερη αξία, επομένως η βέλτιστη θα διαλέξει ένα άλλο, που θα έχει αξία μικρότερη ή ίση με του C_k . Έστω C ένα άλλο ότι το C_j επιλέγεται αργότερα από τη C^* . Αν τα C_k, C_j αλλάζαν σειρά επιλογής από τη βέλτιστη λύση, είτε θα επιλεγόνταν και τα δύο μαζί, διότι θα χωρούσαν στο L και θα επιτρεπόταν από το κριτήριο d με αποτέλεσμα η συνολική αξία να παραμείνει σταθερή, είτε θα επιλεγόνταν μόνο το C_k , με αποτέλεσμα η συνολική αξία να ~~παρέ~~ γινόταν μεγαλύτερη ή ίση από αυτή πριν την ανταλλαγή και αυτό γιατί δε θα χωρούσε το C_j στο L . Επαναλαμβάνοντας αυτή τη διαδικασία, για τα υπόλοιπα αυτοκίνητα της βέλτιστης λύσης, θα καταλήγουμε στη λύση του απρόσκοπτου αλγορίθμου, χωρίς να βελτιώσουμε την αξία της ή να παραβιάσουμε τους περιορισμούς για τα δι. και το L . Επομένως, η C είναι ισοδύναμη ή καλύτερη από τη βέλτιστη και ο αλγόριθμος είναι ορθός.

2. Vi, di, si θετικοί ακέραιοι χωρίς περιορισμούς

Για την επίλυση του προβλήματος της γενικής περίπτωσης, θα χρησιμοποιήσουμε συνδυασμό δυναμικά προγραμματισμού και της δομής Binary Index Trees. Συγκεκριμένα, θα ελέγχουμε όλα τα N αυτοκίνητα και θα αποθηκεύουμε τη βέλτιστη τοποθέτησή τους σε έναν πίνακα BIT. Ο αλγόριθμος διατυπώνεται ως εξής:

- Ταξινομούμε τα αυτοκίνητα σε αύξουσα σειρά d_i και, σε περίπτωση ισοβαθμίας, σε φθίνουσα σειρά v_i .
- Δημιουργούμε ένα πίνακα BIT μεγέθους $L+1$ και αρχικοποιούμε όλες τις θέσεις του σε 0. Το κάθε στοιχείο του πίνακα αντιπροσωπεύει το μέγιστο κέρδος μέχρι τη θέση i . Η απάντηση αντιστοιχεί στο $BIT(L)$.
- Για κάθε αυτοκίνητο με τη σειρά που έχουν ταξινομηθεί, υπολογίζουμε με βάση τη σχέση $t+si \leq d_i$ τις εγγραφές θέσεις που μπορεί να παμάρει. Για τη μέγιστη από αυτές, βρίσκουμε τη μέχρι τώρα τιμή του $BIT.SUM(t)$ και τη συγκρίνουμε με τη νέα τιμή και της προσθέτουμε την αξία v_i . Αν το άθροισμα αυτό είναι μεγαλύτερο από το $BIT.SUM(t+si)$ κάνουμε $BIT.ADD(t+si, \text{old value} + v_i)$, αλλιώς δεν ενημερώνουμε τον πίνακα και συνεχίζουμε την επανάληψη.

Ο αλγόριθμος αυτός διατυπώνεται σε ψευδοκώδικα ως εξής:

```
/*  $d_1 \leq d_2 \leq \dots \leq d_n$  */ (Ταξινόμηση κατά αύξουσα τιμή  $d_i$   
for  $t \leftarrow 0$  to  $L+1$  do και κατά φθίνουσα τιμή  $v_i$  σε  
   $BIT[t] \leftarrow 0$ ; περίπτωση ισοβαθμίας  $d_i$ )  
for  $i \leftarrow 1$  to  $N$  do  
  find max  $t$  such that  $t+si \leq d_i$   
  curval  $\leftarrow BIT.sum(t)$ ;  
  newval  $\leftarrow curval + v_i$ ;
```

```

BIT-add
m ← BIT.sum(t+si);
if newval > m then
    BIT.add(t+si, newval-m);
return(BIT(L));

```

Αναφορικά με την υποδοριστική πολυπλοκότητα του αλγορίθμου, έχουμε για ταξινόμηση των N αυτεινώντων, η οποία υλοτίζεται $N \log N$, ένα iteration από το 1 έως το N , όπου σε κάθε επανάληψη μπορεί να χρειαστεί να υανουμε πράξεις μέχρι και $3N \log L = O(N \log L)$ και την αρχικοποίηση του πίνακα BIT. Η αρχικοποίηση των $L+1$ στοιχείων του πίνακα υλοτίζεται $O(L)$, όπως υποοίμε να την ενσωματώσουμε στο iteration, αρχικοποιώντας τις θέσεις του πίνακα που χρησιμοποιούμε για πρώτη φορά σε 0 το οποίο γίνεται σε $O(1)$ και να "συμπάτουμε" έτσι την αρχικοποίηση με τις υρήσεις του πίνακα. Επομένως, ο αλγόριθμος έχει υποδοριστική πολυπλοκότητα $O(N \log N + N \log L)$.

Αναφορικά με την ορθότητα του αλγορίθμου, έχουμε τα εξής:

- Εξετάζουμε τα αυτεινώντα σε αυξανσα σειρά d_i και θρίνωση σειρά v_i σε περίπτωση ισοθωρίας, με αποτέλεσμα να τοποθετούνται πρώτα τα αυτεινώντα με τους αυστηρότερους περιορισμούς και τη λήιστη αμία.
- Η χρήση της δομής BIT που σε κάθε στοιχείο αποθηκεύει τη συνολική αμία μέχρι το στοιχείο αυτό είναι υατάλληλη για την αναπαράσταση του διαστήματος $[0, L]$ και όλων των υποδιαστημάτων του.
- Κάθε φορά που εξετάζουμε ένα νέο αυτεινώντο, η απόδοση για το αν θα το τοποθετήσουμε στη θέση υάπολου άδδου παίρνεται με βάση την αναδρομική σχέση $R(i, 1) = \max \{ R(i-1, 1-s_i) + v_i, R(i-1, 1) \}$ εξασφαδίζοντας υάθε φορά τη βελτιστη επιλογή.
- Με δυναμικό προγραμματισμό εξετάζουμε και τα N

αυτομίντα, ελασθεαδίζοντας τη βέλτιστη λύση,
χρησιμοποιώντας κάθε φορά την βέλτιστη επιλογή.

Επομένως, ο αλγόριθμος δίνει πάντα τη βέλτιστη
λύση και είναι ορθός.